MAGIC ETL · BUILD SPEC

Rate Card Actuals ETL

Every dataset the Rate Card Tracker reads, built from the assignment table alone and compared against the uploaded pay bands. Build them in order; the second one is the one that matters.

SOURCE	COLUMNS KEPT	TARGET GRAIN
Assignment table	**~55 of ~200**	**One row per assignment**

OUTPUTS
4, two optional

What the app reads

four outputs, two of them optional

OUTPUT	GRAIN	SOURCE	USED FOR	STATUS
`rate_card_client_directory`	One row per client	assignment	Client Matching candidates	BUILT
`rate_card_actuals_assignment`	One row per assignment	assignment	Every actuals report	Build next
`rate_card_title_directory`	Title × client	assignment	Title Matching candidates	With the title matcher
`rate_card_actuals_summary`	Title × geo × year	dataflow 2	Fallback if dataflow 2 runs large	Probably not needed

> **The invoice table is not needed for any of this.** The assignment table has no spend column — spend exists only on the invoice table — but none of the planned reports require it. Dataflow 2 derives pay_dollars from rate × hours, which is what weighting an average and ranking titles by materiality actually need.

KNOW THE LIMIT OF THAT FIGURE

pay_dollars is *computed pay*, not *invoiced spend*. It excludes overtime, expenses, MSP fees and tax adjustments, and it is pay-side rather than bill-side, so it will not reconcile against finance. Label it as pay wherever it surfaces and nobody will mistake the two. If a report is ever asked to tie to a finance number, that is the point to add an invoice rollup grouped to assignment_unique_key — and not before, since joining that table un-grouped multiplies every assignment by its invoice line count.

Settle the grain first

everything downstream branches on this

`assignment_unique_key` implies one row per assignment, but `calendar_reporting_month_date`, `current_date` and `reporting_date_ends` are all present — the signature of a monthly snapshot. Run this before building anything.

```
SELECT COUNT(*)                                    AS rows,
       COUNT(DISTINCT assignment_unique_key)       AS assignments,
       COUNT(DISTINCT calendar_reporting_month_date) AS months
FROM <assignment_table>
```

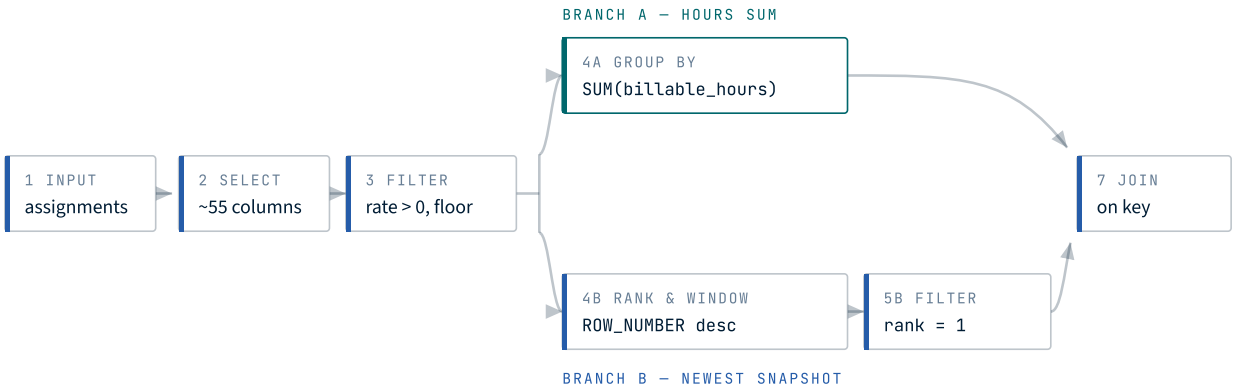
`rows = assignments`

Already at assignment grain. Skip tiles 4A through 7 and add the one aliasing formula in their place.

`rows > assignments`

Monthly snapshot. Build the two branches — rates and hours need opposite treatment.

The fork, in one picture



Two bookkeeping tiles are left out to keep the shape readable: 6B drops Branch B's single-month hours before the join, and tiles 8 and 9 (Add Formula, Output DataSet) continue in a straight line after it. The fork is the only part that isn't obvious.

Dataflow 1 — client directory BUILT

Recorded for completeness. A few hundred rows; feeds the app's Client Matching tab.

#	TILE	CONFIGURATION
1	Input DataSet	Assignment table
2	Select Columns	`rsr_client_code, account_name, country, active_client, billable_hours, calendar_reporting_month_date`
3	Filter Rows	`active_client` is true
4	Group By	Group on `rsr_client_code, account_name, country` · `SUM(billable_hours)` → `hours`, `MAX(calendar_reporting_month_date)` → `latest_month`
5	Output DataSet	`rate_card_client_directory` · Replace

> **Export order for the app:** `rsr_client_code, account_name, country, hours, latest_month`. Paste it straight into the Client Matching tab — the parser takes tab or comma separated, header row optional.

> **Why MAX works here but not in dataflow 3:** calendar_reporting_month_date is a date, and billable_hours is numeric. Group By offers MIN and MAX on those two types only.

Dataflow 2 — assignment actuals what the app reads

1

INPUT DATASET

The assignment table

2

SELECT COLUMNS

Cut to ~55 columns — do this before anything else

On a ~200-column table this is the single largest performance win in the whole dataflow.

```
assignment_unique_key, req_unique_key, assignment_id,
calendar_reporting_month_date,

rsr_client_code, account_name, program_program_name,
contracting_country, active_client,

country, state_province, city, market, rsr_region,
location_level_2, location_level_3, location_level_4, location_level_5,

title_cleaned, req_title, req_title_classification,
ms_category_1, ms_category_2, soc_broad_group, onet_detailed_occupation,

labor_type, rsr_labor_type, collar_type, specialization,
contingent_vs_sow, req_type,

supplier, supplier_code, preferred_supplier_status, supplier_category,

pay_rate_st_hr, pay_rate_st_hr_initial, pay_rate_st_hr_usd,
bill_rate_st_hr, bill_rate_st_hr_initial, bill_rate_st_hr_usd,
supplier_bill_rate_st_hr, markup, assignment_markup_st_hr,
assignment_markup_st_hr_initial, currency,

max_template_bill_rate_st_hr, max_requested_bill_rate_st_hr,
presented_bill_rate_st_hr,
compliance_vs_max_bill_rate, compliance_vs_max_template_bill_rate,

billable_hours, positions_requested_cnt,

assignment_original_start_date, assignment_start_date,
assignment_end_date, assignment_original_end_date, assignment_status,

current_tenure_cd, planned_tenure_cd, time_to_fill_cd,

pay_rate_flag, bill_rate_flag, randstad_sourceright_flag
```

DO NOT SELECT

`worker` · `worker_id` · `consolidated_worker_id` · `cand_id` · `cand_unique_key`

Individual pay attached to a named person is a materially more sensitive dataset than aggregate rates by job title, this feeds a browser app, and no report needs it. If you ever want a distinct-people count, aggregate it here and emit the number.

3

FILTER ROWS

Three conditions, and one you should not add

CONDITION	WHY
`pay_rate_st_hr > 0`	A zero or null rate cannot be compared to a band. Swap in `pay_rate_flag` once its meaning is confirmed.
`assignment_original_start_date` ⩾ earliest card year − 1	Bounds the pull. With cards from 2025, use `2024-01-01`.
`contingent_vs_sow = 'Contingent'`	**Only if** the card is contingent-only. Skip it if the card covers SOW too.

DO NOT FILTER ON ASSIGNMENT_STATUS

Closed assignments are the completed record. Open ones are the only rows still correctable — and they are absent from the invoice table entirely, which is half the reason to build on this table instead.

If the grain check said one row per assignment

Skip tiles 4A through 7. Insert a single Add Formula tile in their place — `hours_to_date = billable_hours` — so both paths converge on the same schema and everything downstream reads `hours_to_date` either way.

BRANCH A Hours accumulate across snapshots, so they sum.

4A

GROUP BY

Total the hours per assignment

GROUP BY	`assignment_unique_key`

AGGREGATE	`SUM(billable_hours)` → `hours_to_date`
	`MIN(calendar_reporting_month_date)` → `first_seen_month`
	`COUNT(*)` → `snapshot_months`

> **snapshot_months** is a cheap sanity check. An assignment with 14 snapshots and 40 total hours is telling you something is wrong upstream.

BRANCH B Rates, status and dates describe current state, so take the newest — never an average.

4B

RANK & WINDOW

Number the snapshots, newest first

FUNCTION	`ROW_NUMBER`
PARTITION	`assignment_unique_key`
ORDER BY	`calendar_reporting_month_date` descending
OUTPUT	`snapshot_rank`

> USE ROW_NUMBER, NOT RANK
>
> If two snapshots share the same reporting month for one assignment, RANK gives both rank 1 and you silently double that assignment. If only RANK is available, follow tile 5B with a Remove Duplicates tile on assignment_unique_key.

5B

FILTER ROWS

Keep the newest snapshot only

CONDITION	`snapshot_rank = 1`

This is also the pattern that carries text columns through a grouping operation, which Group By cannot do — see dataflow 3.

6B

SELECT COLUMNS

Drop this branch's hours

REMOVE	`billable_hours`, `calendar_reporting_month_date`, `snapshot_rank`

Hours come from Branch A. Leaving this branch's single-month figure in place invites someone to reach for the wrong one six months from now.

7

JOIN DATA

Rejoin state with hours

LEFT	Branch B — one row per assignment, current state
RIGHT	Branch A — hours
TYPE	**Left join**
KEY	`assignment_unique_key`

Both branches derive from the same filtered set, so the key sets should match exactly. Left rather than inner means a gap shows up as a null you can see, instead of a row that quietly vanishes.

8

ADD FORMULA

Derived measures, computed once here

```
-- The card year in force when the assignment started.
card_year = CAST(YEAR(assignment_original_start_date) AS CHAR)

-- Rate movement on the same assignment: the renewal question.
pay_drift = pay_rate_st_hr - pay_rate_st_hr_initial

pay_drift_pct = CASE WHEN pay_rate_st_hr_initial > 0
                     THEN (pay_rate_st_hr - pay_rate_st_hr_initial)
                          / pay_rate_st_hr_initial
                     ELSE NULL END

-- Pre-multiplied, so weighted averages are a division downstream.
pay_dollars  = pay_rate_st_hr  * hours_to_date
bill_dollars = bill_rate_st_hr * hours_to_date

-- Open assignments are the correctable ones.
is_open = CASE WHEN assignment_end_date IS NULL
                 OR assignment_end_date > CURRENT_DATE()
               THEN 1 ELSE 0 END

extended = CASE WHEN assignment_original_end_date IS NOT NULL
                 AND assignment_end_date > assignment_original_end_date
                THEN 1 ELSE 0 END
```

CARD_YEAR MUST BE TEXT

The app's rateYear is a string parsed out of the spreadsheet column header — "Max Pay - National Average (2026)". An integer year will not match it on join, and the failure is silent: every row simply finds no card.

9

OUTPUT DATASET

rate_card_actuals_assignment

UPDATE	**Replace**, not append
SCHEDULE	After the assignment table updates
BUDGET	Under ~50,000 rows. If it runs larger, tighten tile 3 by client or year before reaching for the summary.

Dataflow 3 — title directory *needed before title matching works*

The candidate list the title matcher scores against — the direct analogue of the client directory. The **measures** are scoped per client, because the card is per client: the same title sits at different rates for different accounts, and a reviewer resolving "Systems Engineer" needs the rates for *their* client. The **taxonomy** is not — SOC and O*NET codes are derived from the title string by a classification engine, so they belong at title grain and get joined on.

> GROUP BY CANNOT CARRY A TEXT COLUMN THROUGH
>
> MIN() and MAX() are offered on numeric and date columns only. For text the aggregations are Count and Distinct Count, neither of which passes the value through — so the taxonomy columns cannot be aggregated at all. The check below decides which of the two ways round it you need.

```
-- Does one title ever carry more than one classification?
SELECT title_cleaned,
       COUNT(DISTINCT onet_detailed_occupation) AS occupations,
       COUNT(DISTINCT soc_broad_group)          AS soc_groups,
       COUNT(DISTINCT ms_category_1)            AS ms_categories
FROM <assignment_table>
WHERE title_cleaned IS NOT NULL AND title_cleaned <> ''
GROUP BY 1
HAVING COUNT(DISTINCT onet_detailed_occupation) > 1
    OR COUNT(DISTINCT soc_broad_group) > 1
    OR COUNT(DISTINCT ms_category_1) > 1
ORDER BY 2 DESC, 3 DESC
```

`returns nothing`

Taxonomy is functionally dependent on the title. Add those columns to the Group By **keys** alongside the other four — they aggregate nothing, add no rows, and the dataflow stays five tiles. Nothing further to build.

`returns rows`

One title carries several classifications, so putting them in the keys would split that title into several rows and divide its hours across them. Build the lookup branch.

> Why that split matters on a matcher screen: a title whose hours landed in two half-size rows looks small enough to skip. That is precisely the wrong outcome for a screen whose job is helping a reviewer judge materiality.

Measures

#	TILE	CONFIGURATION
1	Input DataSet	Assignment table
2	Select Columns	`title_cleaned, rsr_client_code, country, labor_type, assignment_unique_key, billable_hours, pay_rate_st_hr` · plus the taxonomy columns **only if** the check came back clean
3	Filter Rows	`title_cleaned` is not blank · `pay_rate_st_hr > 0`
4	Add Formula	`pay_dollars = pay_rate_st_hr * billable_hours`
5	Group By	Group on `title_cleaned, rsr_client_code, country, labor_type` · plus the taxonomy columns **as further group keys** if the check was clean

Aggregations for tile 5 — every one of them numeric, which is the point:

- `SUM(billable_hours)` → hours, `SUM(pay_dollars)`
- `COUNT DISTINCT assignment_unique_key` → assignments
- `MIN(pay_rate_st_hr)`, `MAX(pay_rate_st_hr)`

Taxonomy lookup — only if the check returned rows

One row per title, carrying the classification the data most recently agreed on. The same Rank & Window pattern as dataflow 2's branch B, which is the general answer to "I need a text column to survive a grouping".

#	TILE	CONFIGURATION
6	Input DataSet	Assignment table — a second input off the same source
7	Select Columns	`title_cleaned, calendar_reporting_month_date, billable_hours, req_title_classification, ms_category_1, ms_category_2, soc_broad_group, onet_detailed_occupation`
8	Filter Rows	`title_cleaned` is not blank
9	Rank & Window	`ROW_NUMBER` · Partition `title_cleaned` · Order `calendar_reporting_month_date` desc, then `billable_hours` desc · Output `taxonomy_rank`
10	Filter Rows	`taxonomy_rank = 1`
11	Select Columns	Drop `calendar_reporting_month_date`, `billable_hours`, `taxonomy_rank`

Ordering by reporting month descending takes the **most recent** classification, on the reasoning that a classification engine's output improves rather than degrades over time. Hours descending is the tiebreak, so two rows in the same month resolve to the one with more work behind it rather than arbitrarily.

Join and output

#	TILE	CONFIGURATION
12	Join Data	Left: tile 5 measures · Right: tile 11 taxonomy · **Left join** · Key `title_cleaned`
13	Add Formula	`pay_rate_weighted = pay_dollars / hours`
14	Output DataSet	`rate_card_title_directory` · Replace

> **Joining on title_cleaned alone is deliberate.** The lookup is at title grain, so exactly one row matches per title however many client rows the measures produced — no fan-out. Had the lookup been built per client as well, every client without that title would have gone null for no reason.

> **Snapshot-safe either way.** The measures only sum hours and count distinct assignment keys, and the lookup picks one row per title, so both give the same answer at assignment grain or monthly snapshot. No grain branch needed here, whatever the check at the top of this document said.

If that check did return rows, it is worth following up on its own account: it means the four categorization columns disagree for some titles, and settling which one is authoritative matters before any of them is used for fallback matching.

> The pay_rate_min / pay_rate_max pair is what makes a reviewer's decision possible: a candidate title whose actual range sits nowhere near the card band is probably the wrong match, however well the strings score.

Dataflow 4 — summary fallback only, and probably not needed

> READ THIS BEFORE YOU START
>
> Magic ETL's Group By has no percentile aggregation. p25 / median / p75 need either a SQL dataflow or PERCENT_RANK in Rank & Window, if your instance offers it. The better answer is usually to keep Dataflow 2 and compute percentiles in the app, where they can be recomputed at any grain — a summary can only ever serve the grain it was built at.

If you build it anyway, emit **each grain as its own row set**. Percentiles and weighted averages cannot be rolled up from other percentiles, so a state-level row set can never produce the national median.

#	TILE	CONFIGURATION
1	Input DataSet	`rate_card_actuals_assignment`
2a	Add Constants	`grain = 'national'`
3a	Group By	Group on `rsr_client_code, title_cleaned, country, card_year, labor_type, grain`
2b	Add Constants	`grain = 'state'`
3b	Group By	The same, plus `state_province`
4	Append Rows	Stack both branches
5	Add Formula	`pay_rate_weighted = pay_dollars / hours`
6	Output DataSet	`rate_card_actuals_summary` · Replace

Aggregations for both Group By tiles

- `SUM(hours_to_date)` → hours
- `SUM(pay_dollars)`, `SUM(bill_dollars)`
- `COUNT DISTINCT assignment_unique_key` → assignments
- `MIN(pay_rate_st_hr)`, `MAX(pay_rate_st_hr)`

> The weighted average has to be a division *after* the Group By — summed products over summed hours. Averaging **pay_rate_st_hr** directly weights every assignment equally, which treats a three-week placement the same as a two-year one.

> **Text columns here too.** Every grouping column in tiles 3a and 3b is a group key, not an aggregation, so the string fields pass through untouched. Only grain needed inventing, and the Add Constants tile supplies it.

Appendix — SQL equivalent of Dataflow 2

For a SQL dataflow instead of Magic ETL. Written with plain subqueries and no window functions or CTEs, so it runs on the older MySQL dialects as well as the newer engines. Verify the date functions against your instance.

```
SELECT
    s.*,
    h.hours_to_date,
    h.snapshot_months,
    CAST(YEAR(s.assignment_original_start_date) AS CHAR)  AS card_year,
    s.pay_rate_st_hr - s.pay_rate_st_hr_initial           AS pay_drift,
    s.pay_rate_st_hr  * h.hours_to_date                   AS pay_dollars,
    s.bill_rate_st_hr * h.hours_to_date                   AS bill_dollars
FROM assignment_src s
INNER JOIN (
    -- current state: the newest snapshot per assignment
    SELECT assignment_unique_key,
           MAX(calendar_reporting_month_date) AS latest_month
    FROM assignment_src
    WHERE pay_rate_st_hr > 0
      AND assignment_original_start_date ⩾ '2024-01-01'
    GROUP BY assignment_unique_key
) latest
    ON  latest.assignment_unique_key = s.assignment_unique_key
    AND latest.latest_month          = s.calendar_reporting_month_date
LEFT JOIN (
    -- hours accumulate across every snapshot
    SELECT assignment_unique_key,
           SUM(billable_hours) AS hours_to_date,
           COUNT(*)            AS snapshot_months
    FROM assignment_src
    WHERE pay_rate_st_hr > 0
      AND assignment_original_start_date ⩾ '2024-01-01'
    GROUP BY assignment_unique_key
) h
    ON h.assignment_unique_key = s.assignment_unique_key
WHERE s.pay_rate_st_hr > 0
  AND s.assignment_original_start_date ⩾ '2024-01-01'
```

SAME TIE RISK AS RANK

The latest join keeps both rows when two share the newest month for one assignment. Confirm with SELECT COUNT(*), COUNT(DISTINCT assignment_unique_key) FROM <output> and add a tiebreak on assignment_id if those differ.

A SQL dataflow sidesteps the text-aggregation limit entirely — plain SQL will happily MAX() a string. If dataflow 3's lookup branch feels like too many tiles, writing that one in SQL instead is a reasonable trade.

Tile names follow Magic ETL; verify them against your instance's version before assuming a tile exists under exactly that name. Column shortlist, join strategy and report backlog are in `references/domo_supplemental_data.md`.